Analýza historických tenisových dát na predikciu výsledku zápasu

Matúš Gajdoš

Úvod

V profesionálnom tenise je presné predpovedanie výsledkov zápasov dôležité pre viaceré zainteresované strany, vrátane trénerov, športových analytikov a stávkových spoločností. Historické údaje o zápasoch poskytujú rozsiahle informácie o výkonnosti hráčov, herných podmienkach a kontexte turnajov, ktoré môžu byť využité na modelovanie a predikciu budúcich výsledkov zápasov.

Cieľom tohto projektu je analyzovať historické údaje o tenisových zápasoch s cieľom predpovedať výsledok zápasu, konkrétne víťazstvo alebo prehru hráča, pomocou dátovo riadených metód a strojového učenia.

Hlavné otázky

Tento projekt sa snaží odpovedať na nasledujúce kľúčové otázky:

- Ktoré historické a kontextové faktory súvisiace s hráčmi a zápasmi majú najväčší vplyv na výsledok zápasu?
- Do akej miery dokážu historické dáta presne predpovedať víťaza zápasu pred jeho odohraním?
- Aký je prínos jednotlivých typov atribútov (rebríček, vek, povrch, forma) k celkovej predikčnej presnosti modelu?
- Je možné vytvoriť model, ktorý poskytuje nielen presné, ale aj interpretovateľné predikcie výsledkov zápasov?

Kritériá úspešnosti

Projekt bude považovaný za úspešný, ak:

- Identifikuje kľúčové faktory ovplyvňujúce výsledok tenisového zápasu
- Vytvorí prediktívny model, ktorý:
  - Dosahuje lepšie výsledky než jednoduchý baseline model
  - Poskytuje interpretovateľné a konzistentné predikcie
- Preukazuje praktickú využiteľnosť prostredníctvom metrík, ako:
  - Presnosť, log_loss a F1-score
  - Stabilný výkon naprieč rôznymi zápasmi

Postup

Analýza dát

V tejto fáze sa skúma štruktúra tenisových dát, dostupné atribúty a ich základné štatistické vlastnosti. Cieľom je identifikovať relevantné premenné pre predikciu výsledkov a odhaliť možné problémy v dátach, ako sú chýbajúce alebo nevyvážené hodnoty.

Predspracovanie dát

Dáta sa vyčistia, transformujú a pripravia na modelovanie, pričom sa vytvára cieľová premenná reprezentujúca result (výhra hráča). Zároveň sa rieši doplnenie chýbajúcich hodnôt, vytváranie nových príznakov, tranformácia datasetu a kódovanie kategórií.

Modelovanie a Evaluácia

Na predikciu výsledkov sa aplikujú vybrané klasifikačné algoritmy a porovnávajú sa s jednoduchým baseline modelom. Modely sú trénované výhradne na predzápasových údajoch a dôraz sa kladie aj na interpretovateľnosť výsledkov.

Modely sa hodnotia pomocou metrík, najmä precision, log loss a F1-score pre triedu výsledkov. Výsledky sa porovnávajú s baseline prístupom a analyzuje sa, v akých situáciách model dokáže úspešne identifikovať výsledkok zápasu.

Analýza dát

V tejto časti sa zameriavam na analýzu vstupných dat. Data pozostavajú z viacerých datasetov (všetky vo formáte csv súboru) z rôznych rokov tenisových ATP turnajov (zaoberám sa teda mužskými zápasmi). Tieto dáta najskôr korektne spojím do jedného väčšieho datasetu, s ktorým následne budem pracovať. Jednotlivé identifikátory z rôznych datasetov sú korektne spracované a nevznikajú žiadne konflikty. Podrobný popis jednotlivých atribútov je uvedený nižšie.

Atribút	**Popis**	**Dátový typ**
`tourney_id`	Jedinečný identifikátor turnaja, kde prvé štyri znaky označujú rok.	object
`tourney_name`	Názov turnaja.	object
`surface`	Povrch kurtu, na ktorom sa turnaj odohráva.	object
`draw_size`	Počet hráčov v hlavnom pavúku turnaja.	int64
`tourney_level`	Úroveň turnaja vyjadrená kódom podľa okruhu (ATP/WTA/ITF).	object
`tourney_date`	Dátum začiatku turnaja vo formáte YYYYMMDD.	int64
`match_num`	Identifikátor zápasu v rámci turnaja.	int64
`winner_id`	Jedinečný identifikátor víťaza zápasu.	int64
`winner_seed`	Nasadenie víťaza na turnaji.	float64
`winner_entry`	Spôsob vstupu víťaza do turnaja (napr. WC, Q, LL).	object
`winner_name`	Meno víťaza zápasu.	object
`winner_hand`	Dominantná ruka víťaza (R, L alebo U).	object
`winner_ht`	Výška víťaza v centimetroch.	float64
`winner_ioc`	Trojznakový kód krajiny víťaza.	object
`winner_age`	Vek víťaza v rokoch v čase turnaja.	float64
`loser_id`	Jedinečný identifikátor porazeného hráča.	int64
`loser_seed`	Nasadenie porazeného hráča.	float64
`loser_entry`	Spôsob vstupu porazeného hráča do turnaja.	object
`loser_name`	Meno porazeného hráča.	object
`loser_hand`	Dominantná ruka porazeného hráča.	object
`loser_ht`	Výška porazeného hráča v centimetroch.	float64
`loser_ioc`	Trojznakový kód krajiny porazeného hráča.	object
`loser_age`	Vek porazeného hráča v rokoch.	float64
`score`	Konečný výsledok zápasu po setoch.	object
`best_of`	Maximálny počet setov v zápase (3 alebo 5).	int64
`round`	Kolo turnaja, v ktorom sa zápas odohral.	object
`minutes`	Dĺžka zápasu v minútach.	float64
`w_ace`	Počet es víťaza.	float64
`w_df`	Počet dvojchýb víťaza.	float64

Atribút	Popis	Dátový typ
`w_svpt`	Počet bodov pri podaní víťaza.	float64
`w_1stIn`	Počet úspešných prvých podaní víťaza.	float64
`w_1stWon`	Počet bodov vyhraných po prvom podaní víťaza.	float64
`w_2ndWon`	Počet bodov vyhraných po druhom podaní víťaza.	float64
`w_SvGms`	Počet gemov na podaní víťaza.	float64
`w_bpSaved`	Počet odvrátených brejkbalov víťaza.	float64
`w_bpFaced`	Počet brejkbalov, ktorým víťaz čelil.	float64
`l_ace`	Počet es porazeného hráča.	float64
`l_df`	Počet dvojchýb porazeného hráča.	float64
`l_svpt`	Počet bodov pri podaní porazeného hráča.	float64
`l_1stIn`	Počet úspešných prvých podaní porazeného hráča.	float64
`l_1stWon`	Počet bodov vyhraných po prvom podaní porazeného hráča.	float64
`l_2ndWon`	Počet bodov vyhraných po druhom podaní porazeného hráča.	float64
`l_SvGms`	Počet gemov na podaní porazeného hráča.	float64
`l_bpSaved`	Počet odvrátených brejkbalov porazeného hráča.	float64
`l_bpFaced`	Počet brejkbalov, ktorým porazený hráč čelil.	float64
`winner_rank`	Rebríčkové umiestnenie víťaza v čase turnaja.	float64
`winner_rank_poin`	Počet rebríčkových bodov víťaza.	float64
`loser_rank`	Rebríčkové umiestnenie porazeného hráča v čase turnaja.	float64
`loser_rank_poin`	Počet rebríčkových bodov porazeného hráča.	float64

Pre predstavu príklad jedného záznamu:

tour-ney_id	surface	draw_size	tour-ney_level	...	tour-ney_date	match_num	loser_rank
2015-339	Hard	28	A	...	20150104	1	220

Dáta teda zahŕňajú popis jednotlivých turnajov a zápasov, kde sú obsiahnuté podania, počet bodov atď. Nedisponujú napríklad výhrou konkrétneho bodu v danom zápase.

Základné charakteristiky dát:

- Časový rozsah dát: 2015-2023
- Zdroj dát: verejne dostupné historické tenisové štatistiky
- Dáta sú na úrovni jednotlivých zápasov

- 49 atribútov a 26920 počet záznamov
- 93041 chýbajúcich hodnôt (7,9%)
- 7 textových, 8 kategorických a 34 numerických atribútov
- Počet duplicitných hodnôt 2941

Konzistencia

Cieľom je overiť, či jednotlivé veci nie sú reprezentované inými hodnotami.

V prvom rade biele znaky. Môžu poškodiť konzistenciu ak sa nachádzajú za, pred alebo medzi jednotlivými hodnotami.

	leading_ws	trailing_ws	multiple_internal_ws	tabs_or_nbsp
score	4	60	0	0
tourney_name	0	27	0	0
winner_name	0	1	0	0

Figure 1: Tabuľka s bielymi znakmi

V tabuľke 1 sú zobrazené problémové atribúty a počet problémových hodnôt. Tieto nekonzistencie vyriešime odstránením bielych znakov, aby odpovedala ostatným hodnotám.

V touney_name je nekonsistencia v mestách, kde sa turnaj odohráva. Zatiaľ, čo pri väčšine názvov je uvedené mesto alebo podrobná identifikácia, existujú turnaje, ktoré majú rovnaké id líšiace sa rokom a reprezentované rôznymi tourney_id.

```
tourney_name tourney_id
  Adelaide 1  2022-8998
  Adelaide 1  2023-2843
  Adelaide 2  2022-9667
  Adelaide 2  2023-8998
  Belgrade 2  2021-9512
   Cologne 1  2020-9404
   Cologne 2  2020-9406
```

Ďalšie nekonzistencia vzniká pri hráčovi a jeho unikátnym match_num identifikátorom. Ako je zobrazené nižšie (po dôkladnej inšpekcií, či sa nejedná o dvoch rôznych hráčov) je jeden hráč s dvoma rôznymi identifikátormi.

```
player_id      player_name
   211776 Martin Landaluce
   212021 Martin Landaluce
```

Ďalej pri winner_entry je ALT vyjadrená aj ako Alt.

Pri hráčoch a ich dominantnej ruke je takisto nekonzistencia. Primárne sa líšia hodnoty od U (unknown) a práve dominantnej ruky. Preto ak pri hráčovi je známa ruka známa, odstránime U.

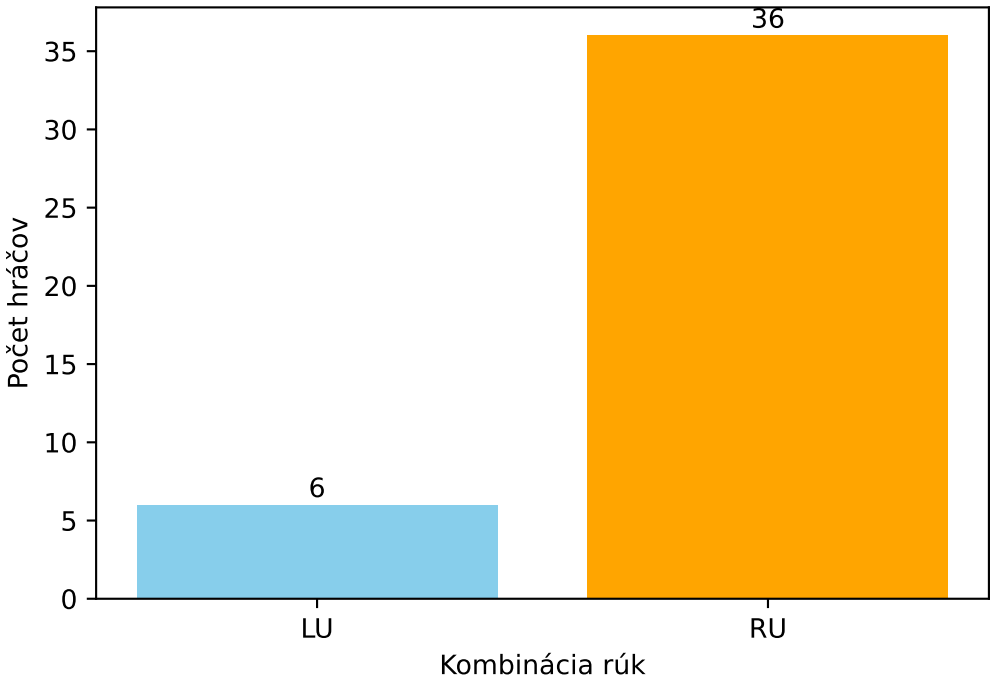

Figure 2: Graf s dominantnou rukou hráčov

Po dôkladnej analýze jednotlivých atribútov a pri ich spojení, ktoré sú spolu súvisiace neboli nájdené žiadne iné nekonzistencie.

Sanity check

Cieľom sanity checku je overiť zmysluplnosť jednotlivých hodnôt atribútov vzhľadom na to, čo majú vyjadrovať. Preto overíme hodnoty a budeme identifikovať, či správne odpovedajú realite. Príkladom môže byť vek, kde očakávame kladné hodnoty a zároveň vyššie ako povedzme 10 rokov. Bolo by zvláštne, keby hralo tenisový zápas dvojročné dieťa na takto vysokej úrovni.

Pri menách hráčov je hráč s číslom v mene (Alejandro Gomez Gb42). Po dohladaní sa jedná o chybu.

Pri hráčoch je vek desatinné číslo. Nejedná sa o chybu, ale zaokrúhlime na celé. Inak vek odpovedá správnym hodnotám, neobjavujú sa chyby.

Pri minutách zápasu sú takisto správne hodnoty. Podozrenie môže priťahovať hodnota 0, ktorá sa v dátach vyskytuje, ale je to možný scénar, kedy sa hráč zraní pred zápasom a podobne.

Po overení jednotlivých hodnôt pri každom atribúte, kde boli skúmané negatívne hodnoty pri číselných atribútoch, nezmyselné kódy krajín, povrchu atď. neboli nájdené žiadne nezmyseľné hodnoty.

Chýbajúce hodnoty

V tejto časti sú rozoberané chýbajúce hodnoty jednotlivých príznakov. Ako je vidieť na grafe 3 pozorujeme, že pri niektorých príznakoch chýba hodnota až v 75% z celkových dátových záznamov. Tieto príznaky (winner_seed, winner_entry, loser_seed, loser_entry) teda zahodíme, bolo by to náročné nahradiť vhodne chýbajúce hodnoty, keď nemáme dostatok nechýbajúcich hodnôt, ktoré by nám pomohly pri doplnení napríklad KNN modelom. Zároveň hodnoty winner_seed sú v určitom kontexte zohľadnené v rankingu. Každý záznam obsahuje chýbajúcu hodnotu, takže mazanie jednotlivých záznamov nie je korektné.

Z grafu 3 je možné vidieť rovnaký počet pri niektorých príznakoch. Je to spôsobené tým, že jednotlivé turnaje počas plynulých rokov nedisponujú týmito informáciami, preto je počet rovnaký.

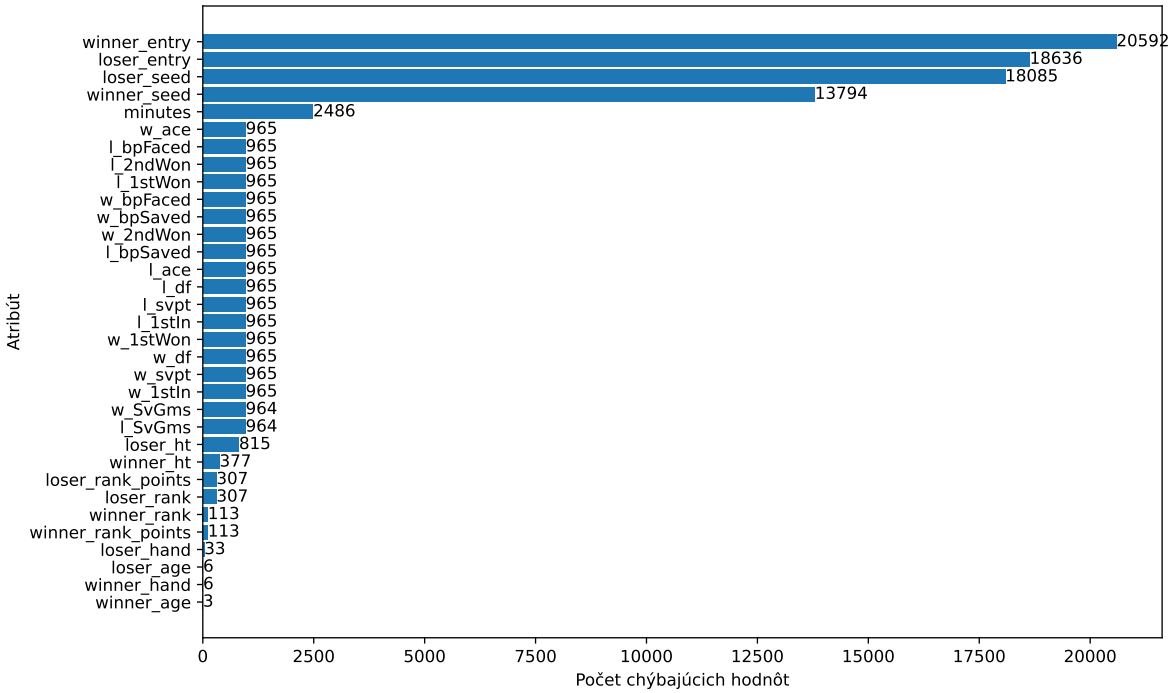

Figure 3: Graf s chýbajúcimi hodnotami

Na doplnenie chýbajúcich hodnôt je použitý KNNImputer. Je to sofistikovanejšia metóda, ako pridanie priemeru. Môže lepšie zohladňovať jednotlivé počty vyhratých bodov a podobne na základe napríklad rankingu.

Datové typy

Na začiatku tejto kapitoly je uviedená tabuľka s atribútmi a ich datovými typmi. Jednotlivé atribúty sú reprezentované správne, kde ale je možná zmena je pri numerických hodnotách, kde väčšinou je uvedený float. Pritom hodnoty sú, s výnimkou veku, celé čísla. V tomto duchu sa pozrieme na jednotlivé typy a pripravíme dataset v sekcií predspracovania dát, kde sa hlbšie ponoríme do ich vhodnej reprezentácie.

Závislosti

Pozrieme sa na určité príznaky, ktoré by mohli priamo ovplyňovať výsledok zápasu. Prvým takýmto príznakom je rank hráča. Chcem zistiť, či vyšší rank je istou zárukou víťazstva.

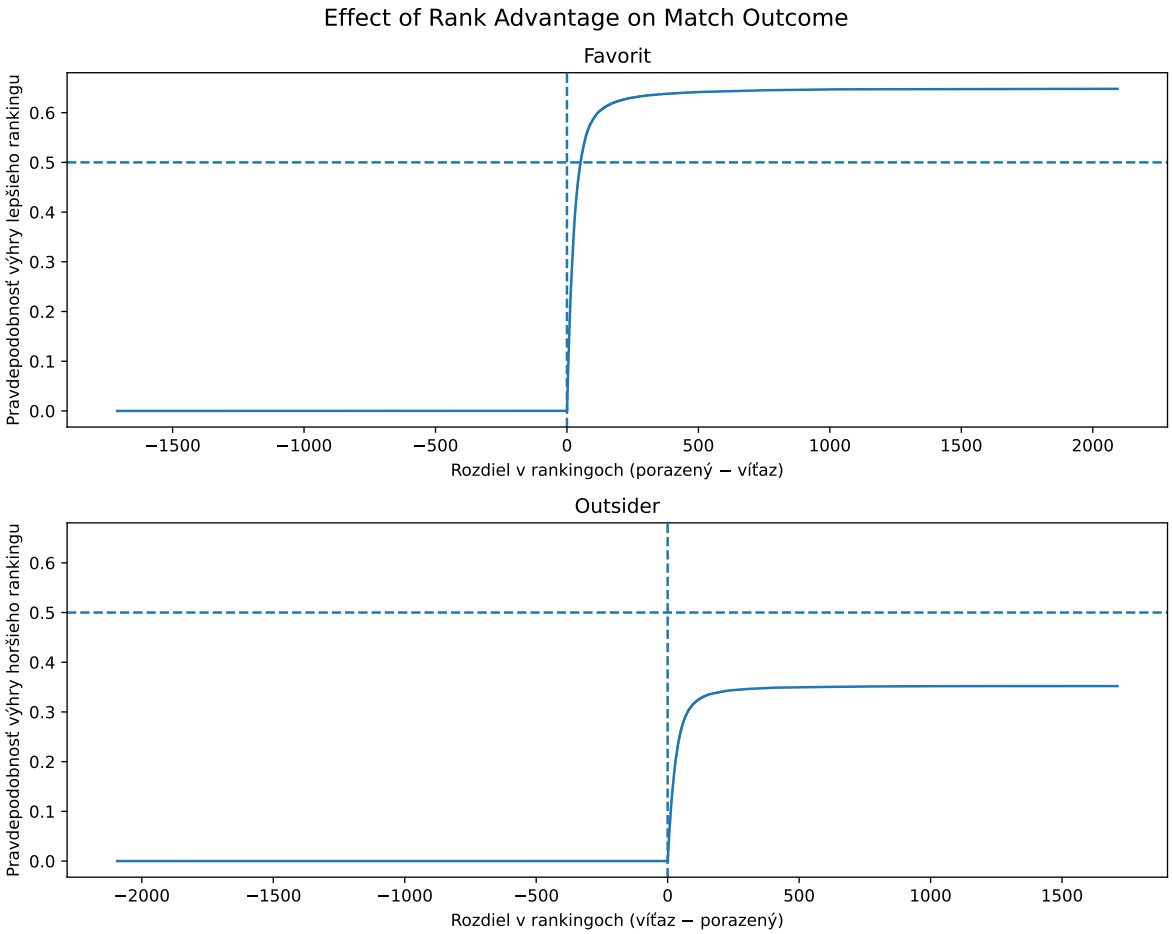

Figure 4: Graf rankingu voči výhre

V grafov 4 sú vykreslené pravdepodobnosti hráčov na základe ich rozdielu v rankoch. Môžme vidieť, že keď má víťaz lepší rank, tak s rastúcim rozdielom rankov (osa x) sa zvyšuje pravdepodobnosť, že vyhrá. Výsledná pravdepodobnost dosahuje okolo 67%, avšak treba si povšimnúť, že menšie rozdiely v rankingoch, v jednotkách až desiatkách nie sú relevantné, pretože pravdepodobnosť výhry nedosahuje ani 50%.

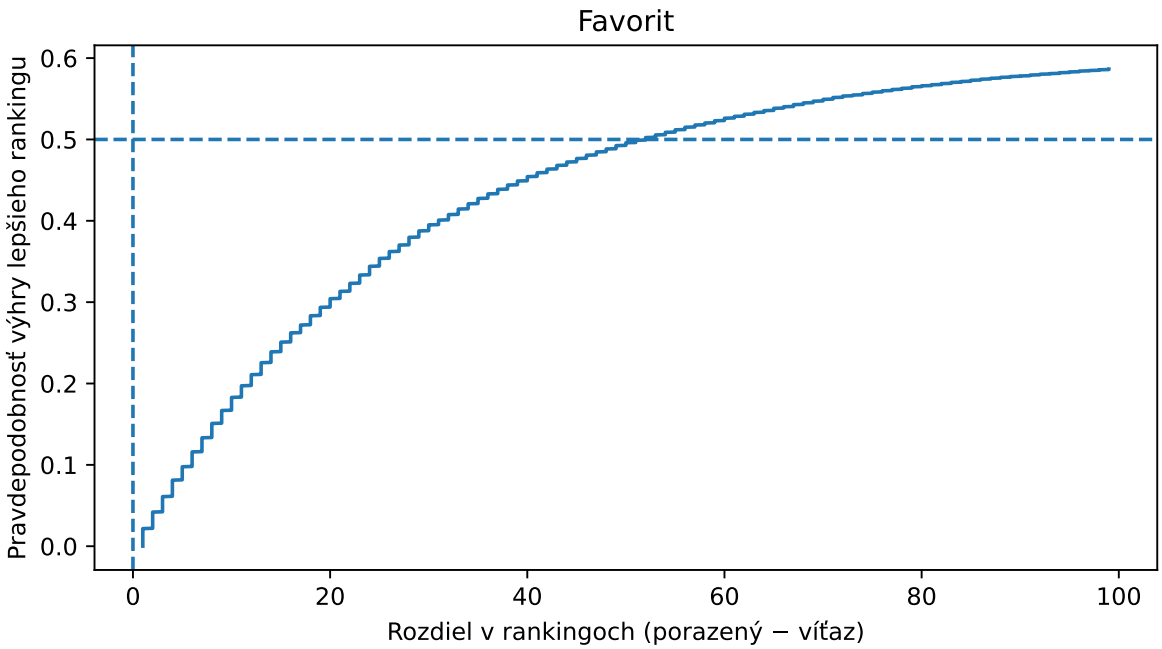

Figure 5: Graf rankingu voči výhre (priblížené)

Z bližšieho pohľadu 5 je vidieť, že až po prekročení rozdielu v rankingoch v hodnote okolo 55 rebríčkov, prekročíme 50% hranicu.

Na druhú stranu, takzvaný prekvapivý zápas, kde outsider (z pohľadu rankingu) vyhrá sa deje v tretine prípadov, čím si sú rankingovo bližší.

Z dát je možné pozorovať rôzne vplyvy jednotlivých príznakov (napríklad úspešnosť prvého podania) na víťaza zápasu. Prepokladáme, že prvé podanie je v plnej sile, aby následný return postavil podávajúceho do výhodnejšej pozície. Pri pohľade na graf 6 je patrné, že víťazi zápasu majú vyššiu úspešnosť vyhratých bodov po prvom podaní (približne o 10%).

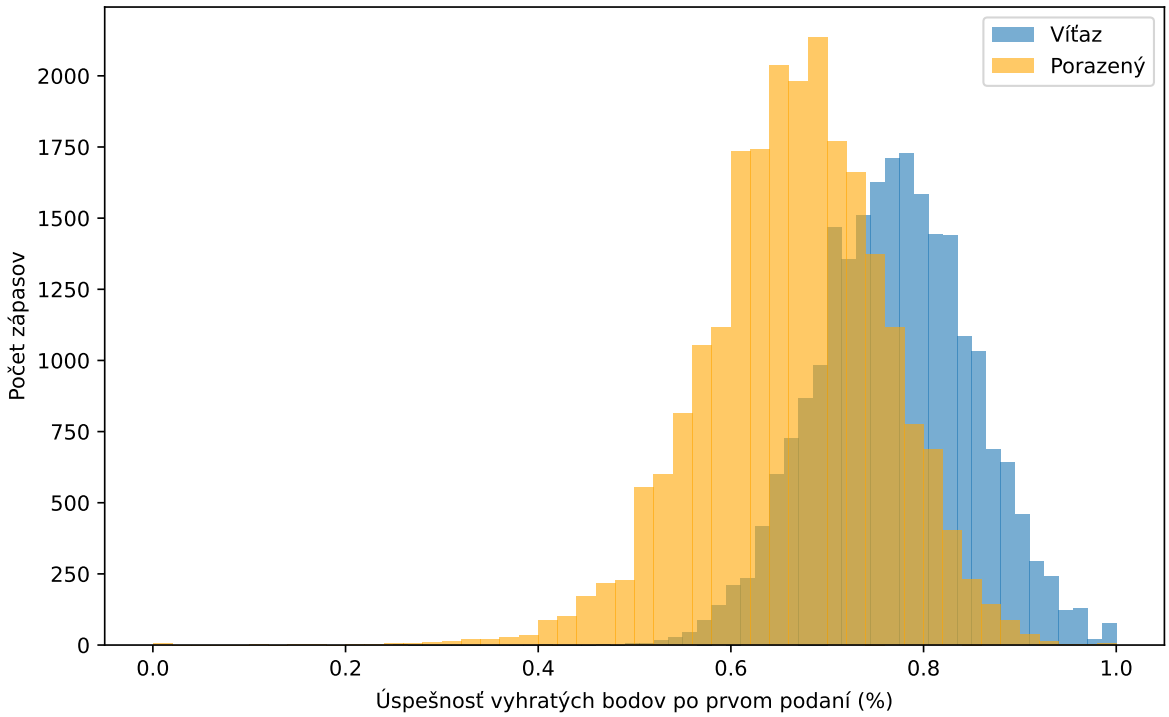

Figure 6: Rozdelenie úspešnosti prvého podania

Avšak pred zápasom tieto dáta zo zápasu nemáme a teda pri predikcií víťaza sa sústredíme na dáta, ktoré budeme pred zápasom poznať. Na tieto hodnoty sa pozrieme v predspacovaní, kde sa tomu podrobnejšie venujem.

Predspracovanie dát

V tejto časti upravíme dataset do požadovanej formy tak, aby sme nad ním mohli následne modelovať. V prvok kroku prevedieme zmeny, ktoré boli spomenuté pri analýze. Konkrétne odstránime nekonzistencie, nezmyselné hodnoty, vysporiadame sa s chýbajúcimi hodnotami a upravíme datové typy.

Následne bude naším cielom vytvoriť nové príznaky, ktoré by potenciálne mohli byť pri modelovaní užitočné (príklad rozdiel v rankingu).

Záverom tejto časti je transformovaný dataset, ktorý je pripravený na modelovanie.

Transformácia

Pôvodný dataset chcem pripraviť na modelovanie, to znamená, že nemôže obsahovať metriky zápasov, ako napríklad počet es, dvojchýb, bodov po prvom podaní atď. Takýmito hodnotami pri predikcií, či daný hráč vyhrá zápas nedisponujeme a budeme sa snažiť zistiť výhercu, čisto na dátach z minulosti. Preto upravýme dataset do formy, kde jednotlivé štatistiky zo zápasu odstránime.

Takisto vhodná reprezentácia dát je potrebná, a preto jednotlivé príznaky transformujeme do vhodného formátu. Čísla, ktoré sú celé reprezentujeme, ako int: vek (zaokruhlené), rank (zaokruhlené), tourney_id, winner_id atď.

Floaty sú napríklad winner_ht, loser_ht, winner_rank_points.

Modely nedokážu pracovať s kategorickými dátami reprezentované stringom. Tieto príznaky, ako meno hráčov, turnajov, povrchu, či iných zakódujeme jednoducho do číselnej podoby, kde každej hodnote priradí svoj identifikátor.

Ďalšia dôležitá tranformácia spočíva v tom, že si dáta namiesto winner a loser pretransformujeme do player_1 a player_2. Dôvodom je, aby sme vedeli skonštruovať vysvetlovanú premennú result (binárna hodnota), ktora vyjadruje, či player_1 vyhral alebo nie.

Po tejto tranformácií sme schopný korektne modelovať nad vysvetlovanou premennou. Hodnoty sú číselné a jednotlivé modely s nimi budú vedieť pracovať.

Nové príznaky

p1_win_streak, p2_win_streak

Pre každého hráča skonštruujem takzvaný win streak (int). Je to celé číslo, ktoré reprezentuje počet vyhratých zápasov v rade podľa dátumu zápasu.

p1_lose_streak, p2_lose_streak

Pre každého hráča skonštruujem lose streak (int). Je to celé číslo, ktoré reprezentuje počet prehratých zápasov v rade podľa dátumu zápasu.

p1_surface_wr, p2_surface_wr

p1_surface_wr je atribút, ktoré reprezentuje percento vyhratých zápasov na povrchu, na ktorom sa aktuálny zápas hrá (float). Takisto je príznak skonštruovaný iba na predošlých zápasoch.

p1_titles, p2_titles

Počet vyhratých turnajov do aktuálneho zápasu.

rank_diff, rank_diff_abs, rank_rel

Po rade odpovedajúce vyššie uvedenému. Rozdieľ hráčov v rankingu vo forme player_1_rank - player_2_rank (int), absolútna hodnota rozdielu rankingu (int) a relatívna hodnota rankingu vypočítaná následovne rank_diff/(player_1_rank + player_2_rank) (float).

age_diff, height_diff, win_streak_diff, lose_streak_diff, surface_wr_diff, titles_diff

Všetky príznaky znamenajú rozdiel medzi dvoma atribútmi. Poporade je to vek (int), výška hráčov (float), win_streak (int), lose_streak (int), surface_wr (float), titles (int).

Modelovanie a Evaluácia

Po fáze predspracovania je k dispozícii výsledný dataset v požadovanom formáte. Dataset obsahuje výhradne numerické atribúty a neobsahuje žiadne chýbajúce hodnoty, čo umožňuje jeho priame použitie v modelovacích algoritmoch. V tejto časti sa zameriavame na modelovanie a predikciu výsledku tenisového zápasu na základe informácií dostupných ešte pred jeho odohraním.

Najskôr je vykonaná analýza korelácie jednotlivých atribútov. Na obrázku 7 je znázornená korelačná matica, ktorá ilustruje vzťahy medzi príznakmi v datasete. Je zrejmé, že niektoré atribúty, napríklad rank a rank_points, vykazujú silnú vzájomnú koreláciu, čo je očakávané, keďže rebríčkové umiestnenie je priamo odvodené od počtu získaných bodov. Rovnako možno pozorovať zvýšenú koreláciu medzi novovytvorenými príznakmi, ktoré sú koncepčne navzájom prepojené.

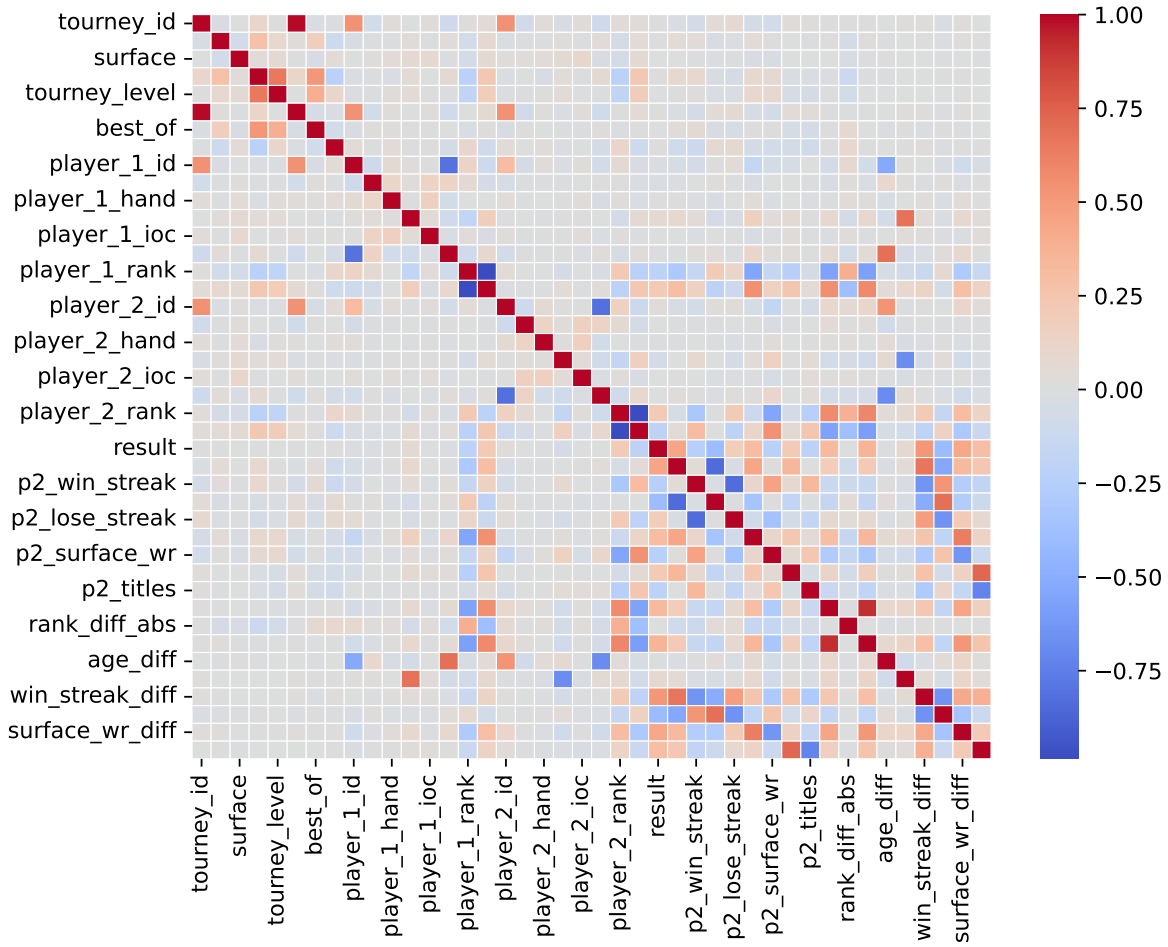

Figure 7: Korelačná matica príznakov

V ďalšom kroku je analyzovaný vzťah jednotlivých príznakov k vysvetľovanej premennej, ktorou je výsledok zápasu. Nižšie sú uvedené príznaky s najsilnejšou koreláciou k cieľovej premennej. Tieto atribúty predstavujú dôležité vstupy pre následné modelovanie, najmä v prípade logistickej regresie, kde koeficienty modelu priamo odrážajú ich vplyv na pravdepodobnosť výhry.

```
result             1.000000
win_streak_diff    0.512831
p1_win_streak      0.444510
surface_wr_diff    0.428829
rank_rel           0.362046
rank_diff          0.319179
p1_surface_wr      0.314880
```

```
titles_diff        0.295758
Name: result, dtype: float64
player_1_rank      -0.212891
p2_surface_wr      -0.236545
p2_win_streak      -0.244722
p1_lose_streak     -0.396556
lose_streak_diff   -0.408858
Name: result, dtype: float64
```

Ako jednoduchý baseline model je použitý náhodný klasifikátor, ktorý simuluje hádzanie mincou. Dataset je najskôr rozdelený na trénovaciu a testovaciu množinu v časovom poradí, aby sa zachovala chronologická konzistentnosť dát. Následne je vyhodnotená presnosť tohto náhodného modelu, ktorá slúži ako minimálna úroveň výkonu, ktorú musia sofistikovanejšie modely prekonať.

```
Presnoť: 0.49
```

Pre ďalšie testovanie modelov, konkrétne logistickej regresie a algoritmu XGBoost, je použitá krížová validácia s časovým rozdelením dát. Trénovacia a testovacia množina sú rozdelené do viacerých podmnožín pomocou metódy TimeSeriesSplit, ktorá zabezpečuje, že model je vždy trénovaný na historických dátach a testovaný na novších pozorovaniach pričom bolo zachované časové poradie zápasov, aby sa predišlo úniku informácií z budúcnosti do minulosti. Tento prístup lepšie simuluje reálne použitie modelu v praxi, kde sú predikcie vždy založené na historických údajoch.

```
Fold 0:
  Train: index 0 to 435
  Test:  index 436 to 871
Fold 1:
  Train: index 0 to 871
  Test:  index 872 to 1307
Fold 2:
  Train: index 0 to 1307
  Test:  index 1308 to 1743
Fold 3:
  Train: index 0 to 1743
  Test:  index 1744 to 2179
Fold 4:
  Train: index 0 to 2179
  Test:  index 2180 to 2615
Fold 5:
```

```
  Train: index 0 to 2615
  Test:  index 2616 to 3051
Fold 6:
  Train: index 0 to 3051
  Test:  index 3052 to 3487
Fold 7:
  Train: index 0 to 3487
  Test:  index 3488 to 3923
Fold 8:
  Train: index 0 to 3923
  Test:  index 3924 to 4359
Fold 9:
  Train: index 0 to 4359
  Test:  index 4360 to 4795
```

Na samotné modelovanie sú použité dva prístupy. Logistická regresia predstavuje lineárny klasifikačný model, ktorý umožňuje jednoduchú interpretáciu vplyvu jednotlivých príznakov na výsledok zápasu. Algoritmus XGBoost je naopak pokročilý stromový model založený na gradientnom posilňovaní, ktorý kombinuje veľké množstvo slabých rozhodovacích stromov a postupne optimalizuje chyby predchádzajúcich modelov. Na ladenie hyperparametrov oboch modelov je použitá metóda GridSearchCV, ktorá umožňuje systematické vyhľadanie optimálnej konfigurácie modelu.

estimator	Pipeline(step..._iter=1000))])
param_grid	{'lr___C': [0.01, 0.1, ...], 'lr___penalty': ['l2']}
scoring	'neg_log_loss'
n_jobs	None
refit	True
cv	TimeSeriesSpl...est_size=None)
verbose	0
pre_dispatch	'2*n_jobs'
error_score	nan
return_train_score	False

copy	True
with_mean	True
with_std	True

penalty	'l2'
dual	False

tol	0.0001
C	1.0
fit_intercept	True
intercept_scaling	1
class_weight	None
random_state	None
solver	'lbfgs'
max_iter	1000
multi_class	'deprecated'
verbose	0
warm_start	False
n_jobs	None
l1_ratio	None

Na hodnotenie modelov boli použité viaceré metriky presnosť (accuracy), F1-skóre a log loss. Okrem samotnej klasifikácie bol kladený dôraz aj na kvalitu pravdepodobnostných výstupov. Hodnotenie prebiehalo pri rôznych rozhodovacích thresholdoch (od 0.4 do 0.6), keďže štandardná hodnota 0.5 nemusí byť optimálna pre všetky modely ani pre daný problém.

V prípade logistickej regresie sa testovala rôzna sila regularizácie, zatiaľ čo pri XGBooste sa optimalizovali parametre ovplyvňujúce komplexnosť stromov, rýchlosť učenia a počet estimátorov. Optimalizácia prebiehala s cieľom minimalizovať log loss, čím sa podporilo lepšie kalibrovanie pravdepodobností.

Výsledky všetkých kombinácií modelov, hyperparametrov a thresholdov boli agregované a vizualizované pomocou heatmapy (obr. 8). Z grafu je zrejmé, že výkon modelov je citlivý nielen na voľbu hyperparametrov, ale aj na rozhodovací threshold. Pri niektorých konfiguráciách dochádza k výraznému zlepšeniu presnosti už pri miernom posune thresholdu od hodnoty 0.5. Zároveň pozorujeme, že každý model výrazne outperformuje baseline model.

```
0%|          | 0/32 [00:00<?, ?it/s]  3%|          | 1/32 [00:05<02:38,  5.12s/it]  6%|
0%|          | 0/4 [00:00<?, ?it/s] 25%|         | 1/4 [00:01<00:04,  1.37s/it] 50%|
```

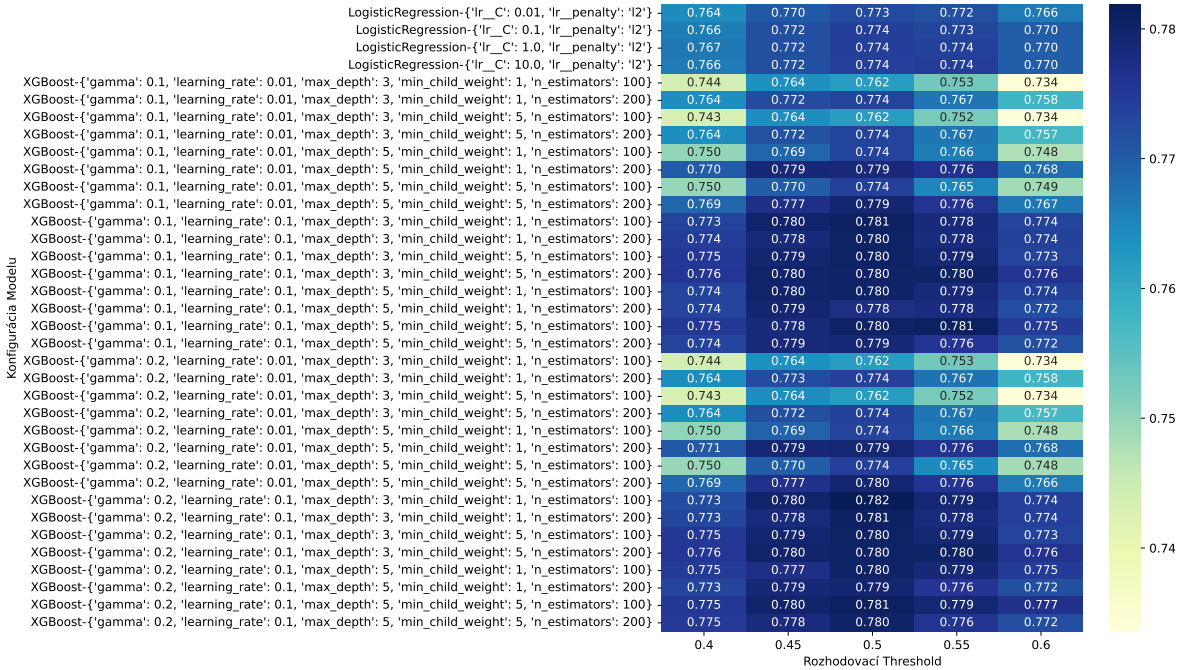

Figure 8: Presnosť cez všetky hyperparametre a threshold (trénovacia množina)

Na výber najlepšieho modelu bola použitá viackriteriálna optimalizácia, kde sa súčasne zohľadňovali tri metriky: presnosť (accuracy), F1-skóre a log loss. Keďže cieľom bolo maximalizovať presnosť aj F1-skóre a zároveň minimalizovať log loss, bola hodnota log loss invertovaná, aby všetky metriky mali rovnaký optimalizačný smer.

Najskôr bola identifikovaná Pareto-optimálna množina modelov, teda modely, ktoré neboli dominované inými kandidátmi vo všetkých sledovaných metrikách. Z tejto množiny bol následne zvolený „najlepší celkový model“ na základe Euklidovskej vzdialenosti od ideálneho bodu (1,1,1), ktorý reprezentuje hypotetický model s perfektnými hodnotami všetkých metrík.

Vybraný model predstavuje najlepší kompromis medzi presnosťou klasifikácie, vyváženosťou tried a kvalitou pravdepodobnostných predikcií. Po jeho výbere bol model finálne natrénovaný na celej trénovacej množine a vyhodnotený na testovacích dátach, kde dosiahol konzistentné výsledky potvrdzujúce jeho stabilitu a praktickú použiteľnosť.

```
Matematicky najlepší celkový model je:
Model: XGBoost
{'gamma': 0.2, 'learning_rate': 0.1, 'max_depth': 5, 'min_child_weight': 5, 'n_estimators':
Vzdialenosť k (1,1,1): 0.5479

CV accuracy: 0.7799770510613884
```

```
CV log_loss: 0.4543824998719006
CV f1: 0.7870509189849519

Test accuracy: 0.78628023352794
Test log_loss: 0.43966260371241433
Test f1: 0.7934716905097723
```

Výsledky vybraných modelov na testovacej množine sú znázornené na grafe 9. Z grafu je zrejmé, že natrénované modely si zachovávajú porovnateľný výkon aj na doposiaľ nevidených dátach a dosahujú podobné hodnoty metrík ako na trénovacej množine. Tento fakt naznačuje, že modely nevykazujú výrazné preučenie a disponujú dobrou schopnosťou generalizácie, čo potvrdzuje ich robustnosť a praktickú použiteľnosť pri predikcii výsledkov zápasov.

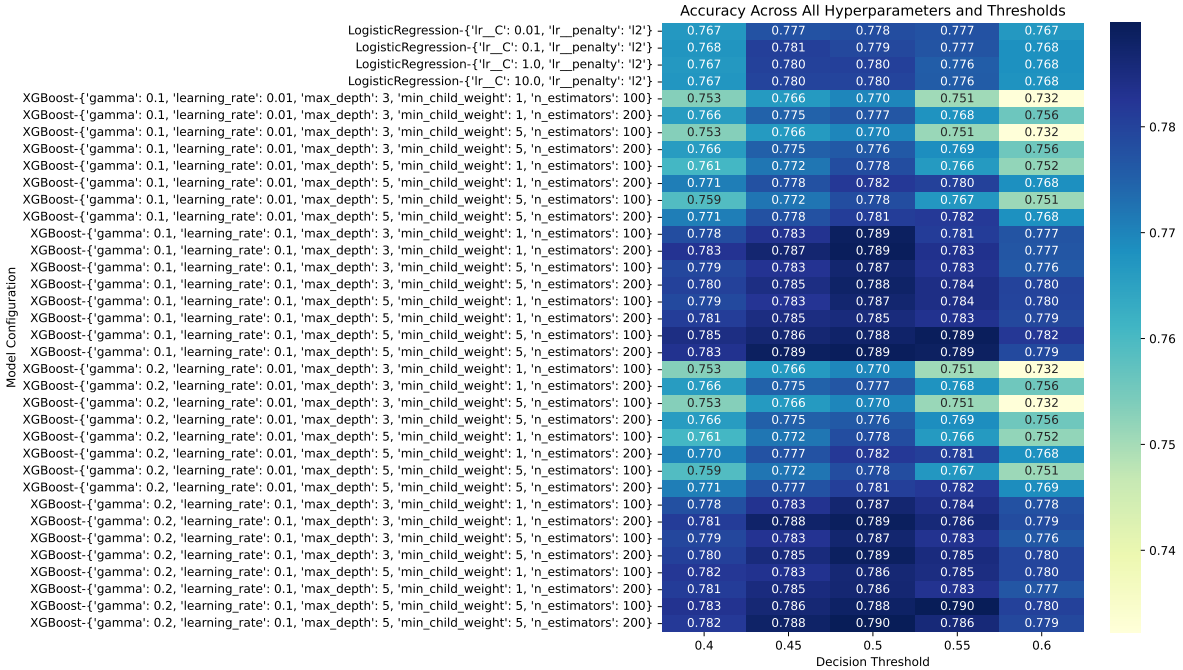

Figure 9: Presnosť cez všetky hyperparametre a threshold (testovacia množina)

Vplyv príznakov

Na lepšie pochopenie správania natrénovaných modelov a interpretáciu ich rozhodnutí bola vykonaná analýza vplyvu jednotlivých príznakov pomocou metódy SHAP (SHapley Additive exPlanations). Táto metóda umožňuje kvantifikovať príspevok každého vstupného príznaku k finálnej predikcii modelu a poskytuje konzistentné vysvetlenie založené na princípoch kooperatívnej teórie hier.

Pre oba vybrané modely XGBoost aj logistickú regresiu boli vypočítané SHAP hodnoty na trénovacej množine. V prípade modelu XGBoost bol použitý TreeExplainer, ktorý je optimalizovaný pre stromové modely, zatiaľ čo pre logistickú regresiu bol aplikovaný LinearExplainer po predchádzajúcej štandardizácii vstupných dát. Tento postup zabezpečuje porovnateľnosť váh jednotlivých príznakov a korektnú interpretáciu lineárneho modelu.

Na obrázku 10 sú znázornené globálne dôležitosti príznakov vo forme stĺpcových SHAP súhrnných grafov pre oba modely. Dĺžka jednotlivých stĺpcov reprezentuje priemernú absolútnu hodnotu SHAP, teda mieru, do akej daný príznak ovplyvňuje rozhodovanie modelu naprieč všetkými pozorovaniami. Z výsledkov je zrejmé, že XGBoost model sa opiera najmä o príznak win_streak_diff súvisiaci s rozdiely medzi počtom po sebe vyhratých zápasov, zatiaľ čo pre model logistickej regresie je dôležitý príznak týkajúci sa počtu titulov. Z pozorovania nám vyplýva, že jednotlivé novovzniknuté príznaky malo zmyseľ vytvoriť a zlepšili tak výkonnosť obidvoch modelov.

Zároveň je možné pozorovať rozdiely medzi modelmi v spôsobe, akým využívajú dostupné informácie. Zatiaľ čo XGBoost zachytáva nelineárne vzťahy a interakcie medzi príznakmi, logistická regresia kladie dôraz na menší počet dominantných, lineárne interpretovateľných premenných. Táto analýza tak nielen zvyšuje dôveru v predikcie modelov, ale zároveň poskytuje cenný vhľad do faktorov, ktoré majú najväčší vplyv na výsledok tenisového zápasu.

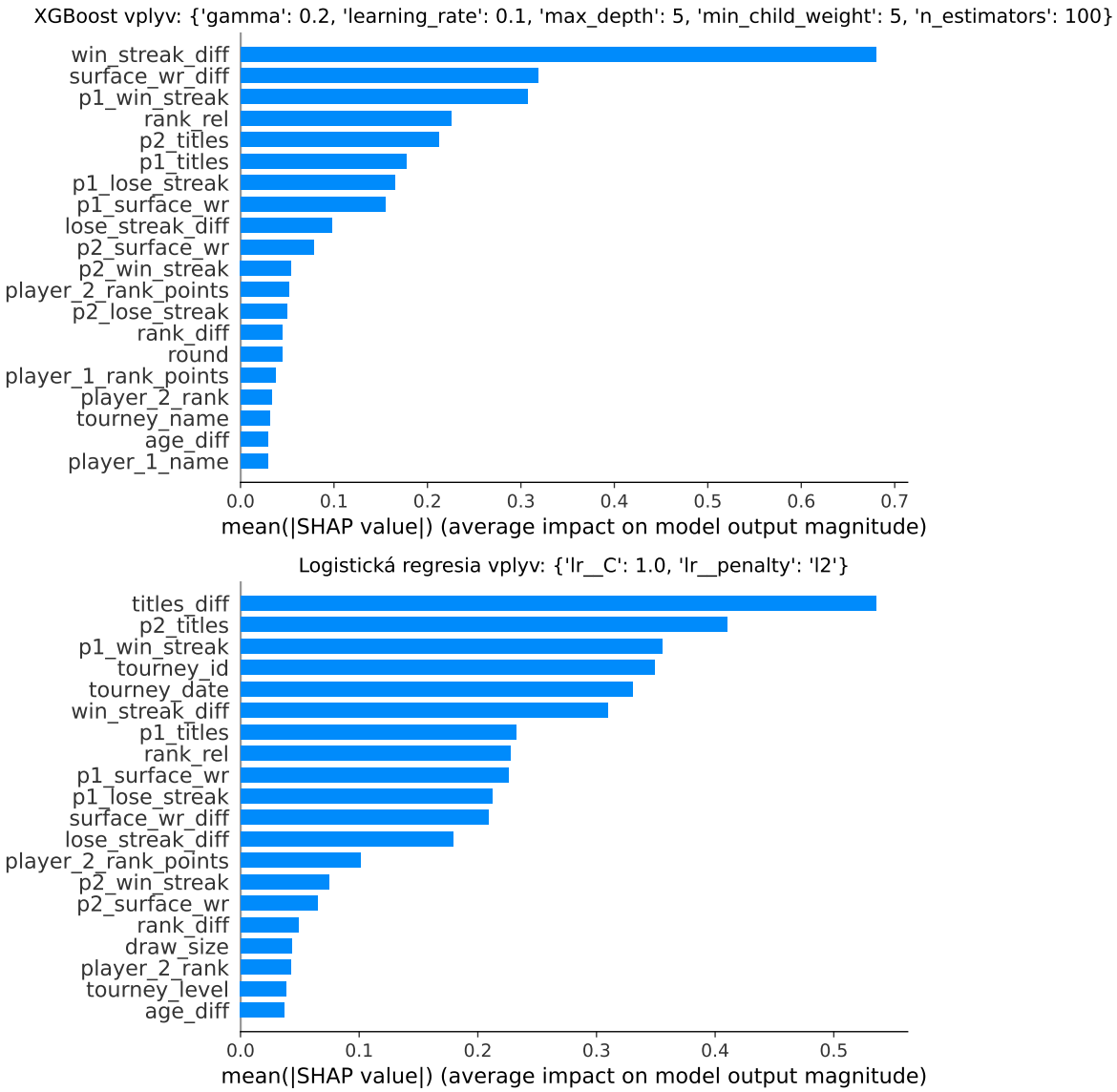

Figure 10: Vplyv jednotlivých parametrov na modely pomocou SHAP

Záver

Cieľom tohto projektu bolo analyzovať historické údaje o tenisových zápasoch a na ich základe vytvoriť modely schopné predpovedať výsledok zápasu ešte pred jeho odohraním. V súlade s metodikou CRISP-DM bol celý proces rozdelený do jednotlivých fáz od analýzy a predspracovania dát, cez modelovanie až po finálnu evaluáciu a interpretáciu výsledkov. Dôraz bol kladený

nielen na samotnú presnosť predikcie, ale aj na robustnosť modelov a ich interpretovateľnosť.

Výsledky ukázali, že oba zvolené modely XGBoost aj logistická regresia sú schopné efektívne generalizovať a dosahovať porovnateľné výsledky na trénovacej aj testovacej množine. To naznačuje, že modely netrpia výrazným preučením a dokážu zachytiť relevantné vzory prítomné v historických dátach. Použitie časovo konzistentnej krížovej validácie zároveň zabezpečilo realistickejšie vyhodnotenie výkonu v kontexte predikcie budúcich zápasov.

Dôležitou súčasťou práce bola interpretácia modelov pomocou metódy SHAP, ktorá poskytla hlbší vhľad do vplyvu jednotlivých príznakov na rozhodovanie modelov. Analýza potvrdila význam novovytvorených príznakov, ako napríklad rozdiel v sérii víťazstiev alebo počet získaných titulov, ktoré sa ukázali ako kľúčové faktory ovplyvňujúce výsledok zápasu. Rozdiely medzi modelmi zároveň poukázali na kontrast medzi schopnosťou XGBoostu zachytávať nelineárne vzťahy a jednoduchosťou a interpretovateľnosťou logistickej regresie.

Z praktického hľadiska práca ukazuje, že historické tenisové dáta môžu byť úspešne využité na predikciu výsledkov zápasov a že vhodná kombinácia feature engineeringu, modelovania a interpretácie vedie k zmysluplným a dôveryhodným výsledkom. V práci sme postupne rozobrali všetky hlavné otázky a predostreli sme odpovede na ne. Zároveň sa podarilo splniť kritéria, kde sme identifikovali kľúčové príznaky pri predikcií, vytvorili sme model s generalizačnou schopnosťou, ktorý mal lepšiu úspešnosť, ako na začiatku vytvorený baseline model. Nakoniec sme odmerali stabitu a úspešnosť daných modelov pomocou metrík na testovacej množine, ktorá zohľadňuje očakávané výsledky pri nových dátach.